

05 — Python dunder methods and special names / 05 — Python 双下划线方法与特殊名称

The question / 原始问题 (Question)

What does dunder mean, and how do I use names such as `__init__`, `__len__`, and `__name__` in Python?
dunder 是什么意思? `__init__`、`__len__`、`__name__` 这些双下划线名称, 在 Python 中怎么使用?

Dunder is short for double underscore. Names such as `__init__` have two underscores at each end. Distinguish special methods from special attributes or module variables: `__len__` is a method, whereas a module's `__name__` is a string.

dunder 是 double underscore 的简称, 指 `__init__` 这种前后各有两个下划线的名称。先区分特殊方法 (special methods) 与特殊属性 / 模块变量 (special attributes / module variables): `__len__` 是方法, 模块的 `__name__` 是字符串。

1. Connect familiar operations to special methods / 1. 常见操作与特殊方法 (Operations and special methods)

When using an object, normally write `len(x)` or `x[0]`. When writing your own class, implement the appropriate special methods to support those operations. These method names are part of Python's protocols; do not invent new dunder names for ordinary helpers.

使用对象时通常写 `len(x)` 或 `x[0]`; 编写自己的类 (class) 时, 才定义相应特殊方法, 使实例支持这些操作。这些名称属于 Python 的协议 (protocols), 普通辅助函数不要自行创造新的 dunder 名称。

Operation 操作 (operation)	Special method 特殊方法 (special method)	Purpose 作用 (purpose)
Initialize an instance 初始化实例	<code>__init__</code>	Set initial attributes 设置初始属性
<code>print(x)</code> , <code>str(x)</code> <code>print(x)</code> 、 <code>str(x)</code>	<code>__str__</code>	Human-readable text 便于阅读的文本
<code>repr(x)</code>	<code>__repr__</code>	Debugging representation 便于调试的对象表示
<code>len(x)</code>	<code>__len__</code>	Number of items 返回元素数量
<code>x[0]</code>	<code>__getitem__</code>	Index or key lookup 按索引或键取值
<code>x + y</code>	<code>__add__</code>	Addition behavior 定义加法行为
<code>x == y</code>	<code>__eq__</code>	Equality comparison 定义相等比较

Operation 操作 (operation)	Special method 特殊方法 (special method)	Purpose 作用 (purpose)
iter(x)	__iter__	Supply an iterator 提供迭代器
x()	__call__	Make an instance callable 让实例可以像函数一样调用

This table is a lookup aid, not a literal expansion of every expression. Operators may involve reflected methods or fallbacks. For example, addition can also consider `__radd__`. Implicit special-method lookup generally uses the object's type, so attaching a method only to an individual instance is not a reliable way to support `len(instance)`.

表格用于识别对应关系，并非每个表达式都只调用这一个方法。运算符还可能使用反向方法（reflected methods）或后备规则，例如加法也可能考虑 `__radd__`。隐式特殊方法查找（implicit special-method lookup）通常从对象的类型查找，因此只给某个实例临时添加方法，并不能可靠地让 `len(instance)` 生效。

2. A small class for scenario profits / 2. 保存情景利润的小类 (ProfitSamples example)

The numbers below are illustrative rounded profits, not a new Case 1 calculation. Save or run [the complete example](#); it requires Python 3.9+ and no external packages.

下面的数值是用于讲解的简化利润，不是重新计算 Case 1。完整示例 需要 Python 3.9+，无需安装额外库。

```
class ProfitSamples:
    def __init__(self, values):
        self.values = list(values)

    def __len__(self):
        return len(self.values)

    def __getitem__(self, index):
        return self.values[index]

    def __repr__(self):
        return f"ProfitSamples({self.values!r})"
```

Now create an instance and inspect it:

创建实例 (instance) 并查看结果：

```
profits = ProfitSamples([0.61, 0.41, 0.71])
print(len(profits))          # 3
print(profits[0])            # 0.61
print(profits)                # ProfitSamples([0.61, 0.41, 0.71])
```

Read our class one method at a time:

逐个理解本例的方法：

- `__init__` stores the supplied values in a list. It initializes an already created instance; object creation is handled by `__new__`. An initializer must not return a non-None value.

`__init__` 将输入保存为列表 (list)。它初始化已经创建的实例；对象创建由 `__new__` 负责。初始化方法不能返回非 `None` 的值。

- `self` refers to the current instance. It is a conventional parameter name, not a keyword or a dunder name. Python supplies it when a bound method is called.

`self` 指向当前实例，是约定的参数名，不是关键字 (keyword)，也不是 dunder。调用绑定方法 (bound method) 时，Python 会自动传入实例。

- `__len__` delegates to the stored list, so this instance reports three items.

`__len__` 调用内部列表的 `len()`，因此本例得到 3。

- `__getitem__` delegates to that list, so index 0 returns 0.61. In this implementation, `profits[:2]` returns a list, not another `ProfitSamples` instance.

`__getitem__` 把索引操作交给列表，所以索引 0 得到 0.61。在这个实现中，`profits[:2]` 返回列表，而不是新的 `ProfitSamples` 实例。

- `__repr__` includes the class name and stored values. The `!r` in the f-string uses `repr(self.values)`. Our class does not override `__str__`, so printing it falls back to this representation.

`__repr__` 在显示结果中包含类名及内部数值。f-string 中的 `!r` 使用 `repr(self.values)`。本类没有重写 `__str__`，所以 `print()` 使用这个表示作为后备 (fallback)。

In Spyder, put a breakpoint on `return len(self.values)` and debug the script. When `len(profits)` reaches the breakpoint, inspect `type(self)`, `self.values`, and `type(self.values)`. This connects the custom instance with the ordinary list it contains.

在 Spyder 中，可在 `return len(self.values)` 上设置断点 (breakpoint)，然后调试脚本。当 `len(profits)` 进入断点时，检查 `type(self)`、`self.values` 和 `type(self.values)`，就能将自定义实例与其中保存的普通列表对应起来。

3. The script entry point: `__name__` / 3. 脚本入口 (Entry point) : `__name__`

Our example ends with:

完整示例最后包含：

```
def main():
    profits = ProfitSamples([0.61, 0.41, 0.71])
    print(len(profits))
    print(profits[0])
    print(profits)

if __name__ == "__main__":
    main()
```

Here `__name__` is a module variable, not a method. Running the file directly gives it the value `"__main__"`, so the demonstration runs. Loading it as an imported module with a different name leaves the guarded call unexecuted. Importing still executes other top-level statements, including class and function definitions; the guard protects only its own block.

这里的 `__name__` 是模块变量 (module variable)，不是方法。直接运行文件时，它为 `"__main__"`，因此执行演示。以其他模块名导入 (import) 时，受保护的 `main()` 调用不执行。导入仍会执行其他顶层语句 (top-level statements)，包括类与函数定义；这个判断只保护自己的代码块。

From the repository root:

在仓库根目录运行：

```
python examples/05_dunder_methods.py
```

Expected output:

预期输出 (expected output) :

```
3
0.61
ProfitSamples([0.61, 0.41, 0.71])
```

For a normal file named `experiment.py`, `import experiment` sets that module's `__name__` to `"experiment"`; package imports can use a dotted name such as `"package.experiment"`. The handbook example's numeric filename is convenient for ordering, so use `importlib` if you want to import it by path rather than writing a normal import statement starting with a digit.

对于普通文件 `experiment.py`，`import experiment` 会让该模块的 `__name__` 为 `"experiment"`；包内导入可以是 `"package.experiment"` 这样的完整名称。手册示例文件名前面的数字便于排序，如需按路径导入，可用 `importlib`，而不是写以数字开头的普通 `import` 语句。

4. Similar underscore patterns have different meanings / 4. 不同下划线形式 (Underscore patterns)

Pattern 写法 (pattern)	Meaning 含义 (meaning)
value	Ordinary name 普通名称
<code>_value</code>	Convention for internal use, not access control 约定为内部使用，不是访问控制
<code>__value</code> inside a class 类内的 <code>__value</code>	Name mangling to reduce subclass name collisions 名称改写 (name mangling)，减少继承中的名称冲突
<code>__value__</code>	Form used for documented special names 文档中定义的特殊名称所使用的形式

For example, a class `Demo` with `self.__value = 1` normally stores it under `_Demo__value`. This is not a security boundary. `__init__` has trailing double underscores and is not a private method.

例如类 `Demo` 中的 `self.__value = 1`，通常会存为 `_Demo__value`。这不是安全屏障。`__init__` 末尾也有双下划线，它不是私有方法 (private method)。

5. What to learn first / 5. 新手学习顺序 (Learning order)

Start with `__init__`, `__repr__`, `__len__`, `__getitem__`, and the `__name__` guard. You do not need a custom class just to calculate profits with NumPy. A class becomes useful when related data and operations should travel together.

优先理解 `__init__`、`__repr__`、`__len__`、`__getitem__` 和 `__name__` 入口判断。使用 NumPy 计算利润并不要求你先写一个类；当相关数据和操作需要组合在一起时，类才更有帮助。

Try predicting `profits[-1]`, `profits[:2]`, and `type(profits.values)`. For this class, the answers are `0.71`, `[0.61, 0.41]`, and `list`. The built-in list provides the indexing rules because our implementation delegates to it.

先预测 `profits[-1]`、`profits[:2]`、`type(profits.values)` 的结果。本例分别得到 `0.71`、`[0.61, 0.41]` 和 `list`。索引规则来自内部列表，因为我们将操作委托（delegate）给了它。

Sources / 资料 (Sources)

See Python's [special-method reference](#)^[1] for protocol details and its [top-level code environment guide](#)^[2] for the entry-point guard. The `ProfitSamples` example and inspection exercise are specific to this handbook.

[Python 特殊方法参考 \(special methods\)](#)^[1] 可用于查阅协议细节；[顶层代码环境 \(top-level code environment\)](#)^[2] 解释入口判断。`ProfitSamples` 示例和检查练习为本手册专门编写。

参考资料 (References)

1. <https://docs.python.org/3/reference/datamodel.html#special-method-names>
2. https://docs.python.org/3/library/__main__.html